Rapport de projet : Étude de rupture entre SQL et Cypher25 sur le Knapsack Path Problem

JIN Christophe – PAN Alexis

Introduction

L'objectif de ce projet est de comparer les performances, la résilience et les limites architecturales d'une base de données relationnelle SQL (PostgreSQL) face à une base de données orientée graphe Cypher (Neo4j). Pour mettre ces deux systèmes à l'épreuve, nous avons choisi de les confronter à un problème algorithmique complexe : le **Knapsack Path Problem**, exécuté sur une topologie de graphe massivement "piégée".

Ce rapport détaille tout le procédé et toutes les approches employées pour répondre au problème.

1. Cadre Théorique : Le Knapsack Path Problem

Nous avons utilisé une variante du problème du Knapsack Problem. Dans la version classique, l'objectif est de remplir un sac ayant une capacité maximale avec des objets possédant un poids et une valeur, afin de maximiser la valeur totale sans déchirer le sac.

Dans notre étude, cette logique est appliquée à un réseau (graphe) :

- **Le Poids maximum :** Devient un **Budget global** que l'algorithme ne doit pas dépasser.
- **Le Poids de l'objet :** Devient le **Coût** de franchissement d'une arête.
- **La Valeur de l'objet :** Devient le **Profit** récolté en visitant un noeud.
- **La Contrainte supplémentaire :** Les éléments ne peuvent pas être choisis indépendamment. Ils doivent former un **Chemin continu** valide d'un point A (Départ) à un point B (Fin).

Le défi algorithmique : Ce problème force le moteur de recherche à adopter un comportement "glouton" (recherche de la maximisation du profit). Si le graphe contient des boucles cycliques offrant un fort profit pour un faible coût, l'algorithme va s’incité à tourner en rond pour accumuler de la valeur jusqu'à épuisement de son budget, provoquant potentiellement une explosion combinatoire.

2. Modélisation et Génération du Graphe Piège

Pour éprouver les limites des moteurs de recherche face à ce problème, nous avons développé un générateur Python créant un graphe orienté de 100102 noeuds et 150101 arêtes (données changeable)

L'architecture de la donnée permet d'exploiter les failles du Knapsack Path Problem en deux structures distinctes :

1. **Le chemin optimal (caché) :** Un chemin allant du Départ vers la Fin, avec des coûts de franchissement normal et des profits minimes, ce qui n'attire pas immédiatement l'algorithme de maximisation.
2. **Les boucles infernales (les pièges) :** Implémentation de 50 000 impasses cycliques qui fait revenir au point de départ. Chaque boucle a un profit immédiat massif pour un coût de déplacement absurde.

Afin de ne pas saturer la mémoire vive lors de la création de ce jeu de données massif, le script Python génère 2 CSV qui stock les arêtes entre les noeuds et les noeuds

3. Stratégies d'Ingestion et Indexation

Les deux bases de données requièrent des approches d'ingestion radicalement différentes.

3.1. Ingestion Relationnelle (PostgreSQL)

- **Technique :** Utilisation de la commande COPY pour l'insertion massif.
- **Indexation vitale :** Création immédiate d'index B-Tree sur les colonnes source et target de la table des arêtes. Sans ces index, la future jointure récursive nécessiterait des balayages séquentiels systématiques sur énormément de ligne, rendant l'exécution du Knapsack Path Problem impossible.

3.2. Ingestion Graphe (Neo4j)

/ ! \ Il faut mettre en commentaire dans le fichier neo4j.conf :

#server.memory.heap.initial_size=128m

#server.memory.heap.max_size=128m

(car plus tard il faudra modifier la mémoire pour pouvoir faire les tests)

- **Technique :** Utilisation de requêtes paramétrées avec l'opérateur UNWIND (Batching). Les données sont envoyées par paquets

optimisés de 10 000 lignes pour éviter la saturation de la couche réseau et de la mémoire transactionnelle du moteur Java.
- **Synchronisation des Index :** Utilisation de CALL db.awaitIndexes() pour garantir la création complète de l'index sur les identifiants des noeuds avant de lier les arêtes.

4. Protocole de Stress-Test et Bridage

Pour comparer équitablement les deux systèmes et observer leur effondrement sous le poids du Knapsack Path Problem, nous avons volontairement bridé les ressources qu'utilise ces 2 bases de données (car cela demanderait énormément de temps pour que les programmes se finissent).

4.1. Mise en place des contraintes (brider les ressources des bases de données)

- **Neo4j (Vulnérabilité In-Memory) :** Modification du fichier **neo4j.conf** pour restreindre la mémoire *Heap* allouée à la JVM, dans les tests nous avons restreint à 128 Mo.
- **PostgreSQL (Vulnérabilité Disque) :** Étant donné que nous ne pouvons pas directement limiter PostgreSQL, nous l'avons manuellement bridé. Limitation agressive de la taille maximale des fichiers temporaires à 100 Mo(temp_file_limit) et réduction de la mémoire de travail à 64 Ko (work_mem) pour forcer le moteur à utiliser le disque dur immédiatement. Un timeout de sécurité a été fixé à 15 secondes.

4.2. Les requêtes concurrentes

- **PostgreSQL :** Utilisation d'une boucle SQL WITH RECURSIVE paramétrée pour accumuler les coûts et les profits. Le moteur effectue une auto-jointure sur les tables et coupe la branche uniquement lorsque e.cout_total + a.cost <= Budget.
- **Neo4j :** Utilisation de Cypher avec l'opérateur d'élagage dynamique allReduce(), vérifiant la condition de budget à chaque saut de noeud.

5. Résultats et Analyse Comparée

Nous avons soumis les deux moteurs à la recherche du meilleur chemin avec des budgets croissants (100, 200, 500, 1000).

5.1. L'effondrement Topologique de PostgreSQL

Pour tout budget supérieur ou égal à 100, PostgreSQL a systématiquement échoué en renvoyant l'erreur suivante : CRASH: temporary file size exceeds temp_file_limit (100000kB)

Analyse : Le langage SQL est attiré par le profit massif des pièges, l'algorithme de jointure récursive s’engouffre dans les boucles infinies. Il a mémorisé chaque passage dans une table temporaire (afin de vérifier le budget total), cette explosion combinatoire des données temporaires a saturé l'espace disque de 100 Mo en quelques millisecondes. PostgreSQL ne peut donc pas résoudre un Knapsack Path Problem sur un graphe cyclique sans l'ajout d'une lourde logique applicative pour mémoriser et interdire les noeuds déjà visités.

5.2. L'immunité Native de Neo4j

Malgré la limite stricte de 128 Mo de RAM, Neo4j a maintenu un temps d'exécution plat et quasi instantané (environ 0.220 secondes chaque test) et a pu tout analyser.

Analyse : Ce résultat met en évidence une propriété fondamentale intégrée au moteur Cypher : **l'Isomorphisme des Relations (Relationship Isomorphism)**. Par défaut, lors de la traversée d'un graphe, Neo4j s'interdit mathématiquement d'emprunter deux fois la même arête au sein du même chemin. Ainsi, dès que l'algorithme (même en cherchant le maximum de gain) est entré dans la boucle piégé, il a pu détecter qu'il avait déjà utilisé cette relation. Il a immédiatement qualifié le chemin d'impasse et l'a élagué, couplé à l'efficacité de l'opérateur allReduce. La RAM n'a donc jamais été saturée.

Conclusion

Nos tests nous a donc permis de démontrer que face à un problème d'optimisation sous contraintes (Knapsack Path Problem) exécuté sur une topologie complexe contenant des cycles rentables, l'architecture relationnelle classique est fondamentalement inadaptée. PostgreSQL ne supporte pas à une explosion combinatoire provoquant l'épuisement de ses capacités de stockage temporaire (Temp File Exhaustion).

À l'inverse, l'architecture orientée graphe de Neo4j ne se contente pas de charger les données en mémoire : elle comprend la topologie. Sa gestion native de l'isomorphisme des relations le protège intrinsèquement des boucles infinies. Cette intelligence structurelle, associée à l'élagage précoce, permet de résoudre le problème en un temps constant et avec des ressources matérielles infimes, là où le modèle relationnel s'effondre.